lazymd -- AGENTS.md

What this is

A ratatui TUI for bidirectional conversion between Markdown, HTML, PDF, and Typst. Users browse the filesystem, preview files in any of the four formats, and convert between them with single-key shortcuts. **All conversions use Rust crates -- never shell out to system tools.**

Architecture

```
src/
  main.rs              -- entrypoint, TUI event loop, panic recovery
  app.rs               -- App state, Mode/Focus enums, key dispatch, process mgmt
  preview.rs           -- preview rendering for md / html / pdf files
  converter.rs         -- crate-based conversion dispatch, tempfile mgmt, error types
  config.rs            -- XDG path discovery, Settings struct, TOML persistence
  fs_utils.rs          -- file listing, format detection, file-tree recursion
  ui/
    mod.rs             -- top-level render(), orchestrates all panels
    layout.rs          -- panel layout calculation (browser + preview + log)
    browser_panel.rs   -- recursive directory tree browser (ratatui List widget)
    preview_panel.rs   -- scrollable preview of the selected file
    log_panel.rs       -- conversion status, errors, process output
    help_popup.rs      -- keybinding reference overlay
```

Conversion matrix (all Rust crates, no system tools)

Direction	Rust crate	Notes
MD ? HTML	`pulldown-cmark`	CommonMark parser. `Parser::new()` + `html::push_html()`.
MD ? PDF	`markdown2pdf`	Lexer + layout engine + printpdf backend. Single crate, no intermediate HTML. `parse_into_file()` or `parse_into_bytes()`.
HTML ? MD	`html2md`	Pure Rust, simple API.
HTML ? PDF	`printpdf` (html feature)	`PdfDocument::from_html()` with inline CSS.
PDF ? MD	`pdf_oxide`	Via `pdf_oxide::api::Pdf::open()` + `to_markdown(page)`. Best-effort, loses some structure.
PDF ? HTML	`pdf_oxide`	Via `pdf_oxide::api::Pdf::open()` + `to_html(page)`. Best-effort.
Typst ? PDF	`typst` + `typst-pdf` + `typst-assets`	Full compilation pipeline. Requires `--features typst`.
Typst ? HTML	`typst` + `typst-html` + `typst-assets`	Full compilation pipeline. Requires `--features typst`.

No system binaries (`md2html`, `pdftotext`, `firefox`, etc.) are used. Every converter is a Rust crate dependency:

```
[dependencies]
pulldown-cmark = "0.13"                     # MD ? HTML
markdown2pdf = "1.6"                        # MD ? PDF (single crate, full pipeline)
printpdf = { version = "0.9", features = ["html"] }  # HTML ? PDF
html2md = "0.2"                             # HTML ? MD
pdf_oxide = "0.3"                           # PDF ? MD / PDF ? HTML
ratatui = "0.29"                            # TUI framework
crossterm = "0.28"                          # Terminal backend
serde = { version = "1", features = ["derive"] }
toml = "0.8"
dirs = "6"
```

```
tempfile = "3"                              # Tempfiles for intermediate conversions
# Typst support (optional, behind "typst" feature)
typst = { version = "0.15", optional = true }
typst-pdf = { version = "0.15", optional = true }
typst-html = { version = "0.15", optional = true }
typst-assets = { version = "0.15", optional = true }
typst-layout = { version = "0.15", optional = true }
typst-library = { version = "0.15", optional = true }
```

Preview rendering strategy

Each format renders differently in the terminal preview panel:

Format	Strategy
Markdown	Rendered as plain text (source). Future: use `pulldown-cmark` events to add basic terminal formatting (colored headings, bold).
HTML	Tags stripped for terminal display via simple tag stripper (`strip_html_tags()`).
PDF	Text extracted page-by-page via `pdf_oxide::api::Pdf::open()` ? `to_text(page)`. Binary/image preview out of scope for v1.
Typst	Rendered as plain text (source). Same as Markdown preview.

Error handling pattern

- `converter.rs` defines a `ConversionError` enum with variants: `UnsupportedFormat`, `ConversionFailed`, `Io`
- All conversion functions return `Result<T, ConversionError>`
- The TUI should display these errors in the log panel, never panic
- `markdown2pdf` errors are wrapped via `.map_err(|e| ConversionError::ConversionFailed(e.to_string()))`

pdf_oxide API (verified)

For PDF reading, use the high-level `pdf_oxide::api::Pdf` struct:

```
use pdf_oxide::api::Pdf;
let mut doc = Pdf::open(path)?;
let page_count = doc.page_count()?;
// Extract from each page:
let text = doc.to_text(page_index)?;      // plain text
let md = doc.to_markdown(page_index)?;    // markdown (best-effort)
let html = doc.to_html(page_index)?;      // HTML (best-effort)
```

Do NOT use internal types like `AutoExtractor` or `TextExtractor` directly. The `Pdf` API is the stable, high-level interface.

markdown2pdf feature flags

`markdown2pdf` has optional features that are **not needed for local file conversion**:

- `fetch` -- enables fetching remote resources (URLs in markdown)
- `svg` -- enables SVG rasterization

For lazymd (local files only), the default feature set is sufficient.

Typst feature gate

Typst support is **optional** and behind the `typst` feature flag. Build with:

```
cargo build --features typst
# or via just:
just build --features typst
```

Without the feature, `.typ` files are still recognized and previewable (as plain text), but conversion to PDF/HTML will return an error message instructing the user to enable the feature.

Typst compilation pipeline

Typst ? PDF/HTML uses the full `typst` compiler crate (v0.15). The pipeline is:

1. Implement the `World` trait (provides library, fonts, source files)
2. Call `typst::compile::<PagedDocument>(&world)` for PDF, or `typst::compile::<HtmlDocument>(&world)` for HTML
3. Export via `typst_pdf::pdf()` or `typst_html::html()`

Fonts are provided by `typst-assets` (embedded Libertine, New Computer Modern, etc.). No system font configuration is needed.

The `World` implementation in `converter.rs` (`typst_impl` module) is minimal:

- Single source file (`main.typ`)
- No external file dependencies
- Current date via `SystemTime`
- All embedded fonts from `typst_assets::fonts()`

Keybindings

Key	Action
`q` / `Ctrl+q`	Quit
`Tab`	Cycle focus (browser ? preview ? log)
`j`/? / `k`/?	Move cursor
`Enter`	Confirm / enter directory / open file
`h`/? / `l`/?	Parent dir / enter dir
`m`	Convert file to **M**arkdown
`h`	Convert file to **H**TML (in non-nav contexts)
`p`	Convert file to **P**DF
`t`	Convert Typst file to PDF/HTML
? / `Esc`	Toggle help / close popup

Distinguish h (nav: parent directory) from h (convert to HTML) by context -- when focus is in the browser panel, h means parent dir; when a conversion keybinding would trigger, use a distinct flow (e.g., h on a previewed file).

Commands

```
just build              # debug build
just build-release      # release build
just run                # run the TUI
just check              # fast compilation check
just test               # all tests
just clippy             # lint (must pass before commits)
just fmt                # format (must pass before commits)
just precommit          # fmt + clippy + test
just doc                # generate docs
just codegraph          # update CodeGraph index
```

CodeGraph

CodeGraph is initialized in this project. Use it for structural code queries:

```
just codegraph          # re-index after changes
just codegraph-status   # view index stats
```

See `.codegraph/` for the index database. The file watcher debounces ~500ms behind writes.

Gotchas

- **Edition 2024** (requires rustc ? 1.85). `cargo fmt` uses 2024 style. `gen` is a reserved keyword. Run `cargo fmt` after any significant edit.
- **ratatui + crossterm**: pin recent stable (`ratatui ? 0.28`, `crossterm ? 0.28`). Same stack as sibling project `lazymagick`.
- **No system tools**. Every converter is a Rust crate. If a crate is missing from `Cargo.toml` the build fails -- no runtime surprises from missing binaries. The tool should give a clear error (not a panic) if a conversion fails at the crate level.
- **`markdown2pdf`** es el crate para MD?PDF. No necesita `printpdf` para esa ruta. `printpdf` solo se necesita para HTML?PDF directo (con feature `html`).

- **`printpdf` html feature**: requiere el flag `features = ["html"]` o `PdfDocument::from_html()` no compila.
- **Testing TUI apps**: test business logic (converter dispatch, file-type detection, config parsing) with standard `#[test]`. TUI integration tests are low priority.
- **Tempfiles**: conversions that pipe through intermediate files (e.g., MD?PDF via temp HTML) should use `tempfile::NamedTempFile` and clean up in all exit paths.
- **PDF preview**: render as text via `pdf_oxide::api::Pdf::open()` ? `to_text()`. Binary or image preview is out of scope for v1.
- **File-type detection**: use extension (`.md`, `.html`/`.htm`, `.pdf`) plus magic-byte fallback (first bytes of PDF: `%PDF`, HTML: < or `<!DOCTYPE`).
- **`just`** command runner is available system-wide with a `justfile`.
- **`printpdf` vs `markdown2pdf`**: `markdown2pdf` handles MD?PDF in a single crate. `printpdf` (with `html` feature) is only needed for HTML?PDF direct. Do NOT use `printpdf` for MD?PDF.
- **pdf_oxide API**: Always use `pdf_oxide::api::Pdf`. Do not use internal `AutoExtractor` or `TextExtractor` directly -- they're internal and may break across versions.
- **Typst World trait**: The `World` implementation in `converter.rs` is minimal and only supports single-file `.typ` sources. External file dependencies (images, includes) will fail with `FileError::NotFound`.